

Vault安全原理_v2

vault实现了信息安全所要求的三要素, 称为CIA

[机密性](#) (Confidentiality)

机密性 (Confidentiality) 确保资料传递与存储的隐密性, 避免未经授权的用户有意或无意的揭露资料内容。

[数据完整性](#) (Integrity)

数据完整性 (Integrity) 代表确保资料无论是在传输或存储的生命周期中, 保有其正确性与一致性。

[可用性](#) (Availability)

在信息安全领域, 可用性 (Availability) 是成功的信息安全项目应具备的需求, 意及当用户需透过信息系统进行操作时, 资料与服务须保持可用状况(能用), 并能满足使用需求(够用)。

网络安全管理框架三要素, 称为3A

认证 (Authentication)

认证用来识别访问网络的用户的身份, 判断访问者是否为合法的用户。

授权 (Authorization)

授权是指对不同用户赋予不同的权限, 限制用户可以使用的服务。依照实际需求给予实体适当的权限, 一般建议采最小权限 (Least privilege), 意即仅给予实际作业所需要的权限, 避免过度授权可能造成的信息暴露或泄漏。

纪录 (Accounting)

用来记录用户使用网络服务过程中的相关操作, 内容项目包含量测 (Measuring)、监控 (Monitoring)、报告 (Reporting) 与日志案 (Logging), 以便提供未来作为审核 (Auditing)、计费 (Billing)、分析 (Analysis) 与管理之用, 主要精神在于收集用户与系统之间交互的资料, 并留下轨迹纪录。

以下说明Vault是如何满足信息安全三要素和网络安全管理三要素

机密性

数据安全:

任何通过 Vault 保存在存储后端的数据都必须是安全的, 不会被窃听。这意味着所有落盘数据都必须加密。

因此Vault 对向后端发出的所有请求使用安全屏障(Security Barrier)。安全屏障使用具有 96 位随机数 [Galois Counter Mode \(GCM\)](#) 的 256 位[高级加密标准 \(AES\)](#) 密码自动加密 Vault 的存储的所有数据。随机数是为每个加密对象随机生成的。

安全屏障(Security Barrier)的实际运行流程:

Vault初始化时会先产生主密钥 (Master Key) 和相应的分片

```
3barrierKey, barrierKeyShares, err := c.generateShares(barrierConfig)
```

安全屏障实际是有一个结构体 主要存储了seal状态和主密钥与加解密密钥的绑定关系

```
type AESGCMBarrier struct {
    // ...
    sealed bool // 是否解封

    keyring *Keyring // 主密钥与加解密密钥的绑定关系
    initialized atomic.Bool // 是否初始化
}

// 用于主密钥和加解密密钥的管理
type Keyring struct {
    masterKey []byte // 用于解封的主密钥
    keys      map[uint32]*Key // 加解密密钥
    activeTerm uint32
}
```

安全屏障Barrier 初始化过程:

```
func (b *AESGCMBarrier) Initialize(ctx context.Context, key, sealKey []byte,
reader io.Reader) error {
    // ...
    // 使用操作系统随机源生成一个随机加解密密钥
    encrypt, err := b.GenerateKey(reader)
    if err != nil {
        return errwrap.Wrapf("failed to generate encryption key: {{err}}", err)
    }

    // 创建一个keyring用于绑定主密钥和加解密密钥的关系
    keyring := NewKeyring()
    keyring = keyring.SetMasterKey(key)
    keyring, err = keyring.AddKey(&Key{
        Term: 1,
        Version: 1,
        value: encrypt,
    })
    if err != nil {
        return errwrap.Wrapf("failed to create keyring: {{err}}", err)
    }

    // 将加解密密钥使用主密钥AES加密后存储到storage中
    err = b.persistKeyring(ctx, keyring)
    if err != nil {
        return err
    }

    // ...
}
```

Barrier安全屏障实际使用流程为:

1. 从存储后端storage backend 取出加密过的加解密密钥
2. 使用用户输入的主密钥master key 解密
3. 组成keyring结构体, 在内存中存储加解密密钥
4. 后续操作使用该加解密密钥负责加解密往来于Vault server 和storage backend 的机密

内存安全:

Vault 的数据在传输时或是落盘时都是加密的, 但是在内存中仍然保有敏感数据的明文和用于加解密的私钥。应通过禁用虚拟内存来最小化数据泄露风险, 以防止操作系统将敏感数据换出到磁盘。

Vault 默认会尝试将其虚拟地址空间锁定到 RAM 中, 以禁用交换到磁盘的功能, 这不仅要求该进程以 root 权限运行, 还要求系统能够支持该mlock()功能。

存储安全:

尽管存储后端的数据是加密的, 但具有最高控制权的攻击者可以通过修改或删除密钥或机密数据来导致数据损坏或丢失。

因此对存储后端的访问应仅限于 Vault, 以避免未经授权的访问或操作。

通信方面:

客户端与 Vault 的通信, 以及从 Vault 到其存储后端, 或 Vault 集群节点之间的通信应该是安全的, 即使通信被拦截, 窃听者也只能访问加密数据。

- 客户端使用TLS来验证服务器的身份并建立安全的通信通道。
- 集群内 Vault 实例之间的所有服务器之间的流量 (即高可用性、企业复制或集成存储) 都使用相互验证的 TLS 来确保传输中数据的机密性和完整性。节点在加入集群之前通过解封质询或一次性激活令牌进行身份验证。每个备用节点使用身份验证的私钥 (ECDSA-P521) 和自签名证书, 建立连接到主节点的经过相互验证身份的 TLS 1.2 连接。当备用节点收到来自客户端的请求时, 请求被序列化, 通过这个受 TLS 保护的通信通道发送, 并由主节点执行。
- Vault 和 存储后端consul 通过 TLS 安全加密的 HTTP 协议和携带 consul 令牌在两者之间进行通信。

完整性

任何篡改已存储的数据都是可检测的, 并导致 Vault 中止事务处理。

从存储后端读取的数据都会经由安全屏障(Security Barrier)解密交由 Vault 服务使用, 当从安全屏障读取数据时, 在解密过程中验证 GCM 身份验证标签以检测数据有无篡改。

GCM可以提供对消息的加密和完整性校验, 它结合了计数器模式 (Counter Mode, CTR) 和 Galois 字段上的操作以提供数据加密和数据完整性的保证。GCM 特别适合对需要高效加密和认证的应用。为了保证安全, GCM 模式下每次加密操作需要使用一个唯一的初始向量 (IV) 或 nonce。

GCM广泛用于需要高性能和高级安全保障的领域如

- 网络通信协议, 如 TLS/SSL。
- 存储系统, 确存储数据的机密性和完整性。

- 非常适合需要并行处理的大数据加密任务。

可用性

Vault 支持多服务器部署模式以实现高可用性。此模式通过运行多个 Vault 服务器来防止服务中断, 降低一台机器或一个进程宕掉时的破坏性, 保证其在短时间宕机时能保障Vault的高可用。

在HA模式下运行时,vault服务器有两种状态: 备用节点和活跃节点, 多个vault服务器节点共享一个存储后端, 在某一时刻只有一个节点处于活跃, 其他节点处于热备用状态。

活跃节点按通常的方式运行并处理所有请求。备用节点不处理请求, 而是将请求重定向到活动节点。这个过程中, 如果活动节点是密封 (seal, 失败或丢失连接, 其中一个备用节点将接管其功能并成为活动节点。

目前支持高可用模式的存储后端有几种, 包括 Consul、ZooKeeper 和 etcd。使用支持高可用的数据存储后端时, 会自动启用高可用模式。

vault通过问责制和身份验证机制来满足网络安全管理3A要求的记录, 授权, 认证

问责制

启用审计日志, 所有的请求与响应的机密都必须在客户端接收到任何机密材料之前被记录下来。

每行审计日志都是一个JSON 对象。 `type` 字段指定了对象的类型。目前有两种类型: `request` 和 `response`。该行包含一次请求和响应的所有信息。所有敏感信息在被记录进审计日志之前首先被哈希。使用 HMAC-SHA256 加盐进行散列, 目的是使机密在审计日志中不以明文形式存在。

Vault 可以与多种不同的审计日志存储的集成, 启用多个审计日志存储, Vault 会将审计日志发送给所有这些存储。这使得我们不仅可以拥有冗余副本, 还可以拥有第二个副本, 以防第一个副本被篡改。

身份验证

解封密钥机制

解封密钥机制是对Vault安全使用的最基本设置, Vault 服务启动后, 默认处于封印状态, 在未解封时, Vault 无法执行任何操作, 必须进行“解封” (Unseal) 处理, 获得与后端存储数据相应的主密钥 (Master Key) 后才能正常工作。

Vault 支持通过 [Shamir 算法](#)实现多人解封, Shamir 算法允许我们将主密钥拆分为多个分片或多个部分。分片的数量和所需的阈值是可配置的, 但默认情况下, Vault 会生成 5 个分片, 必须提供其中的任意 3 个才能重建主密钥。通过使用分片算法, 我们避免了对主密钥持有者的绝对信任, 并完全避免存储主密钥。只能通过重建分片来检索主密钥。这些分片对向 Vault 提出任何请求没有用处, 只能用于解封。

解封密钥也需要严格保密, 通常会将其存储在冷存储中, 以防止未经授权的访问和泄露。只有在必要时才使用解封密钥。

策略机制

策略确切地定义了客户端可以访问哪些秘密以及它们可以使用它们执行哪些操作, 所有的请求都必须按照相关的安全策略进行。

Vault策略使用 HCL 或是 JSON 语法编写, 描述了一个用户或是应用程序允许访问 Vault 中哪些路径。

Vault 使用严格的默认拒绝执行策略。这意味着除非相关策略允许给定操作，否则它将被拒绝。每个策略都指定了授予 Vault 中路径的访问级别。

身份验证方法被用来认证连接到 Vault 服务的用户或是应用程序的身份信息。一旦验证通过，身份验证组件会返回一组当前身份适用的策略信息。Vault 接受一个通过认证的用户，并返回一个可供将来使用的客户端令牌。

Token认证机制

Token机制是对Vault具体功能进行权限控制,是在Vault 解封后使用,Token可以与策略相绑定来进行权限控制。

在每个请求中，客户端都会提供此令牌。然后 Vault 使用它来检查令牌是否有效并且没有被吊销或过期，并根据关联的策略生成 ACL(访问控制列表)。

Vault初始化是会生成根Token,根Token是 Vault 中的最高权限凭据,类似于 Unix 系统上的 root 用户,拥有此令牌的用户可以在 Vault 解封后进行任何操作，包括创建和删除机密、管理策略、配置身份验证等。因此，根令牌需要严格保密，并只在必要时进行使用。

Token的持有者可创建新的Token，这些新Token将被创建为原Token的子Token,父Token可以指定子Token 的权限和过期时间,当撤销某个Token时，其所创建的子Token，以及子Token 的子Token 都会被一并删除。

每个非根Token都有一个与之关联的有效期 (TTL)，这是自令牌的创建时间或上次续约时间（以较近者为准）以来的当前有效期,与之相关可以对Token的有效期进行续订、吊销。

身份认证机制

Vault 中的身份验证是根据内部或外部系统验证用户或机器提供的账户密码的过程。

身份可以与策略相绑定来进行具体的权限控制。

在客户端可以与 Vault 交互之前，必须先使用相关身份验证方法进行身份验证。身份验证后，会生成Token。该Token在概念上类似于网站上的会话 ID。

身份也有与之相关的租约。这意味着我们必须在给定的租约到期后重新进行身份验证才能继续访问 Vault。